SSISDE function reference

Inputs and outputs of all 23 Python files in data_generation/ and models/.

Notation. S or X = state path; t = time grid; dt = time step; N = sample count; T = duration; S0 = initial state; seed = random seed. W is simulated Brownian motion, BQ is recovered Q-noise, and BP is recovered physical Brownian motion. A trailing underscore marks fine-grid arrays in the experiments. Tuples below follow the code's return order.

data_generation/

parameters.py

generate_noise: input N, T, seed. Returns **(t, dt, dW, W)**: time grid, step, Brownian increments and cumulative Brownian path.

processes.py

mu and **sigma**: input state x, drift/diffusion label and coefficients; return the corresponding coefficient values. **simulate_process**: input dW, dt, S0, drift, diff and coefficients; returns **(S_, mu_exact_, sigma_exact_)**. Coefficients: r, sigm, kappa, theta, w1-w4, K, beta; K here is a population-model coefficient.

simulation.py

simulate: input callable functions mu and sigma, plus N, T, S0, seed. Returns **(t, dt, S, W, mu(S), sigma(S))**: time grid, step, state, Brownian path, actual drift and actual diffusion.

constant.py

mu(x, r) returns constant drift r; **sigma(x, sigm)** returns constant diffusion sigm. **generate_data**: input N, T, S0, seed, r, sigm; returns the same six outputs as simulate.

quadratic.py

mu(x) returns $0.3x^2 + 0.7x$; **sigma(x)** returns $0.1x^2 + 0.5x$. **generate_data**: input N, T, S0, seed; returns the same six outputs as simulate.

ou.py

mu(x, kappa, theta) returns kappa(theta - x); **sigma(x, sigm)** returns sigm. **generate_data**: input N, T, S0, seed, kappa, theta, sigm; returns the same six outputs as simulate.

double_well.py

mu(x) returns $0.5x - x^3$; **sigma(x)** returns $1 + 0.25x^2$. **generate_data**: input N, T, S0, seed; returns the same six outputs as simulate.

coarse_graining.py

coarse_grain: input S_, BQ_, W_, mu_exact_, sigma_exact_, sigma_pred_, stride M and coarse count K. Returns **(sigma_pred, mu_exact, S, BQ, W, sigma_exact)**, sampling every M-th entry.

models/stochastic_sindy.py

StochSINDy(...) creates a model. Inputs set polynomial degrees (mu_deg, si_deg), penalty (pen), cross-validation (use_cv, k, alphas, rhos, use_1se), and coefficient thresholds (thr1, thr2).

fit(X, BP, dt, alpha_mu, alpha_si) returns the fitted model itself. **mu(x)** and **si(x)** return drift and diffusion predictions at x. **show()** prints fitted equations and errors.

plot_cv(output_prefix) shows cross-validation plots; **plot_size(count_bias, output_prefix)** shows model-size plots. **plot_results(X, BP, BQ, W, mu_true, si_true, output_prefix)** compares fitted functions and noise; true functions are optional. Plot methods return None and save PNGs where implemented (plot_cv saves for elastic net).

Estimation and recovery

models/ | Remaining 14 files

Key distinction. Diffusion regression fits sigma^2; its positive square root gives sigma_pred. sigma_prime, sigma_2prime and sigma_3prime are derivatives with respect to the state, not time.

quadratic_variation.py

compute_quadratic_variation(W) returns the scalar sum of squared path increments. **estimate_quadratic_variation(S_, t)** returns **(QV, dQV, diff_method)**: cumulative quadratic variation, its time derivative and the differentiation object.

diffusion.py

fit_diffusion_model: input S_, t, dQV and settings threshold, nu, max_iter, tol. Returns a **fitted PySINDy model of sigma^2**, using a degree-4 polynomial library with a constant term.

diffusion_quadratic.py

fit_quadratic_diffusion: input S_, t, dQV and threshold, nu, max_iter, tol. Returns a **fitted model of sigma^2**, using a degree-4 library without a constant term.

diffusion_tuning.py

diffusion_model(threshold, nu, max_iter, tol) returns an **unfitted** variance model. **fit_diffusion(S, t, dQV, thresholds, nus, max_iter, tol)** returns **(model, best, results)**: fitted model, selected settings record and all search records.

diffusion_derivatives.py / coarse_diffusion_derivatives.py

diffusion_derivatives(sigma_pred_, S_, diff_method) returns **(sigma_prime_, sigma_2prime_, sigma_3prime_)**. **coarse_diffusion_derivatives(sigma_pred, S, diff_method)** returns the same three derivatives on the coarse grid, without trailing underscores.

q_brownian_motion.py

recover_q_noise(S_, sigma_pred_) returns **BQ_**, the fine-grid recovered Q-noise path.

signature_regression.py

fit_signature(S_, BQ_, t, keep_cols, M): input state, Q-noise, time grid, selected signature columns and window length. Returns **l21**, one fitted signature coefficient per window.

drift_coefficients.py

drift_coefficients(mu_exact, sigma_pred, sigma_prime, sigma_2prime, l21) returns **(mu_0, a, b, b_)**: initial drift and coefficient arrays for Methods 1 and 2. Only mu_exact[0] is used from the actual drift.

drift_recovery.py

recover_drift(S, a, b, b_, mu_0) returns **(mu_pred1, mu_pred2)**, the drift arrays recovered by Methods 1 and 2.

drift_recovery_from_ODE2.py

recover_drift_ode2(S, sigma_pred, sigma_prime, sigma_2prime, mu_0, lambda_0) returns **(mu_pred11, lamda_pred)**, the recovered drift and its spatial slope. mu_0 and lambda_0 supply the initial drift and initial slope.

p_brownian_motion.py / p_brownian_motion_method2.py

recover_p_noise(BQ, mu_pred1, sigma_pred, T) and **recover_p_noise_method2(BQ, mu_pred2, sigma_pred, T)** each return **(BP, dt, t)**: recovered physical noise, coarse time step and time grid, using the respective drift estimate.

noise_recovery.py

recover_noise(BQ, drift, diffusion, dt) returns the **recovered physical noise path**. This helper accepts the time step directly and returns only the path.

Using a function in an experiment

```
from models.drift_recovery import recover_drift
mu_pred1, mu_pred2 = recover_drift(S, a, b, b_, mu_0)
```

Optional arguments keep the defaults defined in the source files. This reference documents the current code; it changes no computations.